

mathbr: Python User Manual

mathbr project

Version 0.6.0

Contents

1	Getting started	2
1.1	Choose a starting point	3
2	Activation functions	3
3	Loss functions	3
4	Descriptive statistics	4
5	Nonparametric estimation	5
6	Hypothesis tests	6
7	Binary classification evaluation	7
8	Probability distributions	9
8.1	Standard normal	9
8.2	Student-t	9
8.3	Chi-squared and F	9
8.4	Uniform	10
8.5	Bernoulli and Binomial	10
8.6	Poisson	11
8.7	Negative binomial and multinomial	11
8.8	Seeded discrete sampling	11
8.9	Seeded continuous sampling	12
8.10	Geometric and discrete uniform	13
8.11	Exponential, Weibull, Laplace, Cauchy, and log-normal	13
8.12	Gamma and Beta	14
8.13	Pareto and Dirichlet	14
8.14	Closed-form distribution fitting	14
8.15	Multivariate normal and Mahalanobis distance	15
9	Gradient-descent models	15
9.1	LinearRegression	15
9.2	LogisticRegression	16
10	Least squares	16
10.1	OLS	16
10.2	WLS	17
11	Regression diagnostics	17

12 Regularized regression	18
13 Econometrics	18
13.1 IV2SLS	18
13.2 FixedEffects	19
14 Time series	19
15 Time-series diagnostics	19
16 Volatility	20
17 Conjugate Bayesian updating	20
18 Right-censored survival analysis	21
19 Worked examples	22
19.1 Machine learning: fit, predict, and evaluate held-out data	22
19.2 Machine learning: compare regularization strengths	22
19.3 Machine learning: binary probabilities and decisions	23
19.4 Statistics: inspect an OLS fit	23
19.5 Statistics: downweight one observation with WLS	23
19.6 Econometrics: instrumental variables	24
19.7 Econometrics: entity fixed effects	24
19.8 Time series: two-variable forecasting	24
19.9 Volatility: compare ARCH and GARCH forecasts	25
19.10 Probability: use a quantile as a cutoff	25
20 API boundaries	25

1 Getting started

`mathbr` is an educational C++17 extension exposed to Python through `pybind11`. It provides numerical functions and small statistical models. Install it with a C++17 compiler available:

```
python -m pip install .
```

For development, run `python -m pip install -e .` and `python -m pytest`. Import the single top-level module:

```
import mathbr
print(mathbr.version)
```

Model classes are at the top level. Numerical functions live in submodules for activations, losses, descriptive statistics, nonparametric estimation, hypothesis tests, evaluation, distributions, regression diagnostics, time-series diagnostics, survival analysis, and Bayesian updating. A feature matrix `X` is a list of rows; `X[i]` contains the features of observation `i`. A target vector `y` has one entry per row. List inputs are copied into C++; there is no NumPy buffer interface yet. Wrong dimensions and invalid numeric values usually raise `ValueError`. Accessing fitted results before `fit()` raises `RuntimeError`. Python may raise `TypeError` when an argument cannot be converted to its declared type.

1.1 Choose a starting point

For a single data vector, begin with the descriptive-statistics section. For a known probability model, use the distributions section to calculate probabilities or cutoffs. For a fitted relationship, choose a model class, call `fit`, and inspect predictions or coefficients. Use evaluation functions on observations held out from fitting to measure predictive performance. Use hypothesis tests or model inference only after checking the assumptions described beside each method. The worked examples near the end combine several parts of the library.

Inputs are ordinary Python lists. For example, `[[1.0], [2.0]]` represents two observations of one feature, while `[1.0, 2.0]` represents one feature vector for a single prediction. Probabilities use a scale from zero to one; percentages use zero to one hundred only where an argument explicitly says `percentile`. A returned p-value is a tail probability under a statistical null model. It is not the probability that the null model is true.

2 Activation functions

Every scalar activation takes `z`, a numeric scalar, and returns a float. The corresponding derivative returns the derivative with respect to `z`. These functions do not generally reject non-finite input.

```
mathbr.activations.sigmoid(z)                # 1 / (1 + exp(-z))
mathbr.activations.sigmoid_derivative(z)
mathbr.activations.tanh_activation(z)         # tanh(z)
mathbr.activations.tanh_derivative(z)
mathbr.activations.relu(z)                   # max(0, z)
mathbr.activations.relu_derivative(z)         # zero at z = 0
mathbr.activations.leaky_relu(z, alpha=0.01)
mathbr.activations.leaky_relu_derivative(z, alpha=0.01)
mathbr.activations.gelu(z)                   # tanh approximation
mathbr.activations.gelu_derivative(z)
mathbr.activations.swish(z)                  # z * sigmoid(z)
mathbr.activations.swish_derivative(z)
mathbr.activations.softmax(z)                # list of probabilities
```

For leaky ReLU, `alpha` is the negative-side slope and is not range-checked. `softmax` accepts a nonempty sequence of numbers, subtracts its maximum for numerical stability, and returns one probability per input. An empty sequence raises `ValueError`.

```
import mathbr
print(mathbr.activations.sigmoid(0.0))
print(mathbr.activations.softmax([1.0, 2.0, 3.0]))
```

3 Loss functions

The two inputs must be nonempty sequences of equal length; otherwise these functions raise `ValueError`. They return a scalar float except for `mse_derivative`, which returns one derivative per prediction. The general losses do not explicitly check every input for finiteness.

```
mathbr.losses.mse(y_true, y_pred)             # mean squared error
mathbr.losses.mse_derivative(y_true, y_pred)  # d(MSE)/d(y_pred)
mathbr.losses.mae(y_true, y_pred)            # mean absolute error
mathbr.losses.rmse(y_true, y_pred)           # root mean squared error
mathbr.losses.logloss(y_true, y_pred)        # binary log loss
```

For `logloss`, labels must be integer 0 or 1 and predictions must be finite probabilities in $[0, 1]$; violations raise `ValueError`. The logarithm clips probabilities internally to avoid taking $\log(0)$. These are loss calculations, not cross-validation tools.

```
import mathbr
print(mathbr.losses.mse([1.0, 2.0], [1.5, 1.5]))
print(mathbr.losses.logloss([0, 1], [0.1, 0.9]))
```

4 Descriptive statistics

The `mathbr.statistics` submodule summarizes finite, nonempty numeric sequences. Invalid input raises `ValueError`. Most functions return floats; `five_number_summary` returns a list. A `ddof` of 1 gives the usual sample variance or covariance; 0 gives a population denominator. Quantiles use linear interpolation at index $p(n - 1)$.

```
mean(x)                median(x)
mode(x)                variance(x, ddof=1)
standard_deviation(x, ddof=1) skewness(x)
excess_kurtosis(x)     quantile(x, p)
percentile(x, p)       interquartile_range(x)
five_number_summary(x) covariance(x, y, ddof=1)
pearson_correlation(x, y)
spearman_correlation(x, y)
kendall_tau(x, y)
covariance_matrix(data, ddof=1)
correlation_matrix(data)
spearman_correlation_matrix(data)
kendall_correlation_matrix(data)
weighted_covariance_matrix(data, weights)
weighted_correlation_matrix(data, weights)
weighted_mean(x, weights)
weighted_variance(x, weights)
weighted_covariance(x, y, weights)
weighted_correlation(x, y, weights)
weighted_quantile(x, weights, p)
log_sum_exp(x)
log_empirical_mgf(x, t)
empirical_mgf(x, t)
```

`quantile` takes `p` in $[0, 1]$; `percentile` takes `p` in $[0, 100]$. Mode counts exact equal values and returns the smallest on a tie. Skewness and excess kurtosis use population central moments without small-sample correction and require positive variance. Correlations require both inputs to vary. `kendall_tau` returns Kendall's tau-b: $(C - D) / \sqrt{(N - T_x)(N - T_y)}$, where C, D count concordant and discordant pairs, N counts all pairs, and T_x, T_y count ties in each coordinate. It uses sorting and a Fenwick tree in $O(n \log n)$ time and $O(n)$ memory. The weighted functions require aligned, strictly positive finite weights and divide by the weight sum; weighted variance and covariance have no degrees-of-freedom correction.

```
import mathbr
s = mathbr.statistics
data = [1.0, 2.0, 2.0, 3.0, 4.0]
print(s.mean(data), s.median(data), s.mode(data))
print(s.five_number_summary(data))
```

```

print(s.variance(data), s.standard_deviation(data))
print(s.weighted_mean([0.0, 2.0], [1.0, 3.0]))
print(s.spearman_correlation([1, 2, 3], [3, 2, 1]))
print(s.kendall_tau([1, 2, 3], [3, 2, 1]))
print(s.weighted_quantile([0, 10, 20], [1, 2, 1], 0.5))
print(s.log_sum_exp([1000, 1000]))
print(s.empirical_mgf([-1, 0, 1], 0.5))
print(s.log_empirical_mgf([1000, 1000], 1))

```

`weighted_quantile` uses the smallest observed value whose cumulative sorted weight reaches `p` of the total; it is a step function, unlike the unweighted interpolated quantile. `log_sum_exp` subtracts the maximum internally to avoid exponent overflow, and requires a nonempty finite vector. The empirical moment-generating function is $\widehat{M}(t) = n^{-1} \sum_i e^{tx_i}$. `log_empirical_mgf` evaluates $\log \widehat{M}(t)$ with max shifting in $O(n)$ time; `empirical_mgf` exponentiates that result and raises `ValueError` if it exceeds finite floating-point range. These functions describe the supplied sample rather than a fitted probability distribution. For matrix functions, `data` is a nonempty rectangular list of observation rows, each with at least one finite value. The returned square matrix follows the column order. `covariance_matrix` uses denominator $n - \text{ddof}$; `correlation_matrix` uses Pearson correlation, `spearman_correlation_matrix` applies average ranks to each column before Pearson correlation, and `kendall_correlation_matrix` applies tau-b to each pair of columns. Correlation matrices require every column to vary. The covariance computation centers each column once, traverses only the upper triangle, and mirrors the result; its cost is $O(np^2)$ for n rows and p columns. Spearman additionally sorts each column in $O(pn \log n)$ time. Kendall uses $O(p^2n \log n)$ time. `weighted_covariance_matrix` and `weighted_correlation_matrix` require one positive finite weight per row with a finite total. They use population normalization by the weight sum, matching the scalar weighted functions. The implementation centers each column once, scales centered values by the square root of normalized weight, and computes only the upper triangle in $O(np^2)$ time and $O(np + p^2)$ memory.

```

print(s.covariance_matrix([[1, 2], [2, 4], [3, 6]]))
print(s.correlation_matrix([[1, 2], [2, 4], [3, 6]]))
print(s.spearman_correlation_matrix([[1, 2], [2, 4], [3, 6]]))
print(s.kendall_correlation_matrix([[1, 2], [2, 4], [3, 6]]))
print(s.weighted_covariance_matrix([[1, 2], [2, 4]], [1, 3]))
print(s.weighted_correlation_matrix([[1, 2], [2, 4]], [1, 3]))

```

These are descriptive, in-sample summaries. They do not provide hypothesis tests, missing-data handling, or uncertainty estimates.

5 Nonparametric estimation

The `mathbr.nonparametric` submodule evaluates an empirical CDF, Gaussian kernel density estimate, or Nadaraya–Watson kernel regression at a vector of query points. Samples must be nonempty and finite; query points may be an empty vector. Bandwidths must be finite and positive. A fixed bandwidth is supplied by the caller; no automatic selection or boundary correction is provided.

```

empirical_cdf(sample, points)
gaussian_kde(sample, points, bandwidth)
nadaraya_watson(x, y, points, bandwidth)

```

The empirical CDF counts observations less than or equal to each point. KDE averages standard Gaussian kernels divided by bandwidth. Nadaraya–Watson predicts a locally weighted mean of

y at each point; it shifts log weights to improve numerical behavior for distant queries. KDE and kernel regression cost $O(nm)$ for n observations and m query points; the empirical CDF sorts once and then uses binary search.

```
import mathbr
n = mathbr.nonparametric
print(n.empirical_cdf([2, 1, 2, 3], [1, 2, 3]))
print(n.gaussian_kde([-1, 1], [0], bandwidth=1))
print(n.nadaraya_watson([-1, 1], [2, 4],
                        [0], bandwidth=1))
```

6 Hypothesis tests

The `mathbr.hypothesis` submodule provides two-sided t-tests. Each call returns a `TTestResult` with read-only float attributes `statistic`, `degrees_of_freedom`, and `p_value`. Inputs must be finite with at least two observations and positive relevant variance; otherwise the call raises `ValueError`.

```
one_sample_t_test(x, null_mean=0.0)
paired_t_test(before, after)
welch_t_test(x, y)
bonferroni_correction(p_values)
holm_correction(p_values)
benjamini_hochberg_correction(p_values)
proportion_z_test(successes, trials, null_p)
chi_square_goodness_of_fit(observed, expected)
one_way_anova(groups)
kruskal_wallis(groups)
chi_square_independence(table)
mann_whitney_u(x, y)
```

The paired test analyzes aligned `before` - `after` differences. Welch's test uses separate sample variances and an approximate Satterthwaite degrees of freedom; it does not assume equal population variances. Exact one-sample and paired small-sample p-values require independent Gaussian observations or Gaussian paired differences. Welch assumes independent samples and uses an approximate reference distribution. The three correction functions return adjusted p-values in input order; Bonferroni and Holm target family-wise error, while Benjamini-Hochberg targets false discovery rate under independence or suitable positive dependence.

```
import mathbr
h = mathbr.hypothesis
one = h.one_sample_t_test([1.0, 2.0, 3.0])
paired = h.paired_t_test([3, 4, 5], [1, 2, 4])
welch = h.welch_t_test([1, 2, 3], [2, 4, 6])
for result in (one, paired, welch):
    print(result.statistic, result.degrees_of_freedom,
          result.p_value)
print(h.holm_correction([0.01, 0.04, 0.03]))
z = h.proportion_z_test(60, 100, 0.5)
chi = h.chi_square_goodness_of_fit([10, 20, 30],
                                   [20, 20, 20])

print(z.statistic, z.p_value)
print(chi.statistic, chi.degrees_of_freedom,
```

```

        chi.p_value)
anova = h.one_way_anova([[1, 2, 3], [4, 5, 6]])
kw = h.kruskal_wallis([[1, 2, 2], [2, 3, 4]])
print(anova.statistic, anova.df_between, anova.df_within,
      anova.p_value)
print(kw.statistic, kw.degrees_of_freedom, kw.p_value)
independence = h.chi_square_independence([[10, 20], [20, 10]])
print(independence.statistic, independence.degrees_of_freedom,
      independence.p_value)
mw = h.mann_whitney_u([1, 2, 3], [4, 5, 6])
print(mw.statistic, mw.z_score, mw.p_value)

```

The proportion test returns `ZTestResult` and needs at least five expected successes and failures. The goodness-of-fit test returns `ChiSquareTestResult` and uses $k - 1$ degrees of freedom only when no distribution parameters were estimated from the tested counts. Small expected bin counts can make its chi-squared approximation unreliable. `one_way_anova` takes at least two nonempty finite groups, with more total observations than groups and positive within-group variation. It returns an `AnovaResult` with read-only `statistic`, `df_between`, `df_within`, and `p_value`. The F statistic is $[SS_B/(k-1)]/[SS_W/(n-k)]$ and takes $O(n)$ time. Classical p-values assume independent Gaussian observations and equal group variances. `kruskal_wallis` accepts at least two nonempty finite groups and pooled variation. It returns a `ChiSquareTestResult` with `statistic`, $k - 1$ degrees of freedom, and asymptotic p-value. Equal values receive average ranks and a tie correction. Centered rank sums avoid subtracting two large, nearly equal terms; sorting dominates its $O(n \log n)$ cost. The chi-squared approximation may be poor with small groups or many ties. `chi_square_independence` accepts a rectangular contingency table with at least two rows and columns, finite nonnegative counts, and positive row and column totals. It computes expected counts from the margins and returns a `ChiSquareTestResult` with $(r - 1)(c - 1)$ degrees of freedom. Expected counts must be positive; small expected counts weaken the chi-squared approximation. It makes two $O(rc)$ passes, one for margins and one for the statistic. `mann_whitney_u` compares two independent nonempty finite samples. It returns `MannWhitneyResult` with `statistic` equal to the U count for the first sample, `z_score`, and two-sided `p_value`. Pooled ties receive average ranks, and the normal-reference variance adjusts for ties. The z-score uses a half-unit continuity correction. This is an asymptotic p-value, not an exact small-sample test. Sorting costs $O(n \log n)$ time and $O(n)$ memory; all-tied data raise `ValueError`.

7 Binary classification evaluation

The `mathbr.evaluation` submodule accepts aligned binary labels, with 1 as the positive class. The confusion matrix has true classes in its rows and predicted classes in its columns:

```
[[TN, FP], [FN, TP]]
```

Undefined precision, recall, or F1 returns zero. All functions reject empty or mismatched inputs, labels outside 0 and 1, and non-finite scores.

```

confusion_matrix(y_true, y_pred)
precision(y_true, y_pred)
recall(y_true, y_pred)
f1_score(y_true, y_pred)
roc_curve(y_true, scores)
roc_auc(y_true, scores)
precision_recall_curve(y_true, scores)

```

```

rmse(y_true, y_pred)
mae(y_true, y_pred)
mape(y_true, y_pred)
log_loss(y_true, probabilities)
k_fold_indices(n_samples, n_splits, seed=0)
calibration_curve(y_true, probabilities, n_bins=10)

```

`roc_curve` returns a `RocCurve` with read-only `fpr`, `tpr`, and `thresholds` arrays. Its first point is (0, 0) at an infinite threshold. `precision_recall_curve` returns a `PrecisionRecallCurve` with read-only `precision`, `recall`, and `thresholds` arrays. Each point corresponds to classifying scores at or above its threshold as positive. Both curves require examples of both classes, group tied scores, and sort thresholds in descending order. ROC AUC integrates the trapezoids between successive ROC points.

```

import mathbr
e = mathbr.evaluation
actual = [0, 0, 1, 1]
scores = [0.1, 0.4, 0.35, 0.8]
predicted = [int(score >= 0.5) for score in scores]
print(e.confusion_matrix(actual, predicted))
print(e.precision(actual, predicted), e.recall(actual, predicted))
print(e.f1_score(actual, predicted), e.roc_auc(actual, scores))
roc = e.roc_curve(actual, scores)
pr = e.precision_recall_curve(actual, scores)
print(roc.fpr, roc.tpr, roc.thresholds)
print(pr.precision, pr.recall, pr.thresholds)

```

At threshold 0.5, this example has two true negatives, one false negative, and one true positive. Precision is 1 because its sole positive prediction is correct; recall is 0.5 because it finds only one of two positives. The ROC AUC is 0.75. Threshold-dependent scores answer a different question from AUC, which uses the ordering of all scores. Evaluate these metrics on held-out observations when judging a trained classifier. Regression metrics take finite, aligned numeric vectors. `rmse` is root mean squared error and `mae` is mean absolute error. `mape` returns a fraction rather than a percentage and rejects zero targets. `log_loss` takes binary labels and probabilities in [0, 1]; a confidently wrong probability of exactly zero or one yields infinity. `k_fold_indices` returns seeded, shuffled validation index lists; use the complementary indices for each training fold. It balances fold sizes to within one observation. `calibration_curve` uses equal-width probability bins, omits empty bins, and returns a `CalibrationCurve` with read-only `mean_predicted`, `fraction_positive`, and `counts` arrays.

```

import mathbr
e = mathbr.evaluation
print(e.rmse([2, 4], [1, 6]), e.mae([2, 4], [1, 6]))
print(e.mape([2, 4], [1, 6]))
print(e.log_loss([0, 1], [0.2, 0.8]))
folds = e.k_fold_indices(8, 3, seed=7)
print("validation folds:", folds)
cal = e.calibration_curve([0, 1, 1, 0],
                          [0.0, 0.2, 0.5, 1.0], n_bins=2)
print(cal.mean_predicted, cal.fraction_positive,
      cal.counts)

```

8 Probability distributions

The functions below are in `mathbr.distributions`. A PDF is a density, a PMF is a discrete probability, a CDF is cumulative probability, and a PPF is the inverse CDF. Log forms help with tiny probabilities. Invalid parameters raise `ValueError`; iterative special-function nonconvergence can raise `RuntimeError`.

8.1 Standard normal

The standard normal has mean zero and variance one. Each `x` must be finite. Each `p` must be finite and in `[0, 1]`. The PPF returns negative or positive infinity at `p = 0` or `p = 1`.

```
normal_pdf(x)          normal_logpdf(x)
normal_cdf(x)          normal_logcdf(x)
normal_ppf(p)
```

All return floats. The PDF can underflow far into a tail; use the log-PDF there. The CDF uses `erfc`, the far lower-tail log-CDF uses an asymptotic expansion, and the PPF uses bisection.

```
import mathbr
d = mathbr.distributions
print(d.normal_pdf(0.0), d.normal_cdf(1.0), d.normal_ppf(0.975))
```

8.2 Student-t

The standardized Student-t functions take finite `x` and finite, positive degrees of freedom `df`. The PPF additionally takes `p` in `[0, 1]` and returns infinities at the endpoints. They do not estimate location, scale, or degrees of freedom.

```
student_t_pdf(x, df)    student_t_logpdf(x, df)
student_t_cdf(x, df)    student_t_logcdf(x, df)
student_t_ppf(p, df)
```

Each returns a float. The CDF uses a regularized incomplete-beta continued fraction; its logarithmic form retains lower-tail precision. The PPF uses bracketing and bisection, so it is costlier than a direct approximation. Invalid arguments raise `ValueError`; failed continued-fraction convergence raises `RuntimeError`. OLS and WLS use Student-t for classical p-values and intervals under their stated error assumptions.

```
import mathbr
d = mathbr.distributions
print(d.student_t_cdf(1.0, 10.0))
print(d.student_t_ppf(0.975, 10.0))
```

8.3 Chi-squared and F

Chi-squared takes finite positive degrees of freedom `df`. F takes two finite positive degrees of freedom, `df1` and `df2`. Both distributions have nonnegative support. Invalid parameters or non-finite `x` raise `ValueError`; numerical nonconvergence can raise `RuntimeError`. Each function returns a float.

```
chi_square_pdf(x, df)    chi_square_logpdf(x, df)
chi_square_cdf(x, df)    chi_square_logcdf(x, df)
chi_square_ppf(p, df)
f_pdf(x, df1, df2)       f_logpdf(x, df1, df2)
f_cdf(x, df1, df2)       f_logcdf(x, df1, df2)
f_ppf(p, df1, df2)
```

p must be finite in $[0, 1]$. Each PPF returns zero at $p = 0$ and infinity at $p = 1$. A density can be infinite exactly at zero when its first degree of freedom is below two. Chi-squared CDF uses incomplete gamma; F CDF uses incomplete beta. Both PPFs use bracketed bisection, and extreme-tail results can be limited by double-precision rounding.

```
import mathbr
d = mathbr.distributions
print(d.chi_square_cdf(2.0, df=4.0))
print(d.chi_square_ppf(0.95, df=4.0))
print(d.f_cdf(1.0, df1=5.0, df2=10.0))
print(d.f_ppf(0.95, df1=5.0, df2=10.0))
```

These distribution functions are mathematical building blocks. They do not automatically produce valid p -values for a fitted model; the model-specific test statistic and assumptions must be established first.

8.4 Uniform

The interval bounds a and b must be finite, satisfy $a < b$, and have finite width. x must be finite and p must lie in $[0, 1]$. All functions return floats.

```
uniform_pdf(x, a, b)          uniform_logpdf(x, a, b)
uniform_cdf(x, a, b)          uniform_logcdf(x, a, b)
uniform_ppf(p, a, b)
```

The PDF is $1/(b - a)$ within the interval and zero outside; the log-PDF is negative infinity outside. The CDF is linear inside, zero below, and one above. The PPF returns a and b at its endpoints. Invalid inputs raise `ValueError`.

```
import mathbr
d = mathbr.distributions
print(d.uniform_pdf(0.0, -1.0, 1.0))
print(d.uniform_cdf(0.0, -1.0, 1.0))
print(d.uniform_ppf(0.5, -1.0, 1.0))
```

8.5 Bernoulli and Binomial

k is an integer outcome and p is a finite success probability in $[0, 1]$. Binomial also takes nonnegative integer trial count n . Out-of-support k has zero PMF; CDFs are zero below and one above support. All return floats. Invalid p or n raises `ValueError`.

```
bernoulli_pmf(k, p)           bernoulli_cdf(k, p)
binomial_pmf(k, n, p)         binomial_cdf(k, n, p)
binomial_ppf(q, n, p)
```

Binomial assumes independent trials sharing one success probability. Its CDF uses the regularized incomplete beta function, avoiding a sum over up to n masses; the numerical iteration is bounded at 1000 steps. The PPF returns the smallest success count with CDF at least q using $O(\log n)$ CDF evaluations. Extreme PMFs can underflow.

```
import mathbr
d = mathbr.distributions
print(d.bernoulli_pmf(1, 0.3))
print(d.binomial_pmf(2, 4, 0.5), d.binomial_cdf(2, 4, 0.5))
print(d.binomial_ppf(0.5, 4, 0.5))
```

8.6 Poisson

The Poisson distribution uses a finite nonnegative **rate** and integer outcomes $k \geq 0$. The PMF is $e^{-\lambda}\lambda^k/k!$; the CDF uses the regularized upper incomplete gamma function and avoids summing $k+1$ masses. For **rate**=0, all mass is at zero. The PPF returns the smallest integer k with CDF at least **q**, using binary search; it returns infinity for **q**=1 when the rate is positive. PPF results beyond the supported 32-bit integer range raise **ValueError**.

```
poisson_pmf(k, rate)      poisson_cdf(k, rate)
poisson_ppf(q, rate)

import mathbr
d = mathbr.distributions
print(d.poisson_pmf(2, 4), d.poisson_cdf(2, 4))
print(d.poisson_ppf(0.5, 4))
```

8.7 Negative binomial and multinomial

The negative binomial distribution counts failures $k \geq 0$ before a positive integer number of successes r , with independent success probability $p \in (0, 1]$. Its PMF is $\binom{k+r-1}{k}p^r(1-p)^k$ and its CDF uses the regularized incomplete beta function $I_p(r, k+1)$, avoiding direct tail summation. The PPF uses an adaptive upper bound and binary search; it returns infinity at **q**=1 when $p < 1$. Quantiles outside the supported 32-bit integer range raise **ValueError**.

```
negative_binomial_pmf(k, successes, p)
negative_binomial_cdf(k, successes, p)
negative_binomial_ppf(q, successes, p)
multinomial_pmf(counts, probabilities)
```

The multinomial PMF uses nonnegative integer category counts and aligned nonnegative probabilities that sum to one within 10^{-12} . A positive count in a zero-probability category has mass zero. Log-gamma coefficients avoid explicit factorials and take $O(c)$ work for c categories. This API supplies a joint PMF; it does not define a multivariate CDF or quantile.

```
import mathbr
d = mathbr.distributions
print(d.negative_binomial_cdf(2, 3, 0.5))
print(d.negative_binomial_ppf(0.5, 3, 0.5))
print(d.multinomial_pmf([1, 1, 1], [0.2, 0.3, 0.5]))
```

8.8 Seeded discrete sampling

The sampling functions below return an integer list of length **count**. Each call creates a local Mersenne Twister generator from **seed**, so equal seeds and parameters reproduce the same sequence with the same C++ standard-library implementation. Exact draws may differ across platforms. Geometric samples count the success trial from 1; negative-binomial samples count failures. The work is linear in output count, apart from each distribution's internal draw cost. Poisson sampling requires a rate within the integer output range.

```
binomial_sample(count, n, p, seed)
geometric_sample(count, p, seed)
poisson_sample(count, rate, seed)
negative_binomial_sample(count, successes, p, seed)
multinomial_sample(trials, probabilities, seed)
bernoulli_sample(count, p, seed)
discrete_uniform_sample(count, a, b, seed)
```

```

import mathbr
d = mathbr.distributions
print(d.binomial_sample(5, 10, 0.3, seed=42))
print(d.geometric_sample(5, 0.5, seed=42))
print(d.poisson_sample(5, 4.0, seed=42))
print(d.negative_binomial_sample(5, 3, 0.5, seed=42))
print(d.multinomial_sample(10, [0.2, 0.3, 0.5], seed=42))
print(d.bernoulli_sample(5, 0.3, seed=42))
print(d.discrete_uniform_sample(5, -2, 2, seed=42))

```

`multinomial_sample` returns one count per category after `trials` categorical draws, preparing the probabilities once. The counts sum to `trials`. It accepts the same finite, nonnegative probabilities summing to one as `multinomial_pmf`. `bernoulli_sample` returns zero/one trials; `discrete_uniform_sample` draws integer values from the inclusive interval `[a,b]`. Both use a local seeded generator for the whole batch.

8.9 Seeded continuous sampling

Each sampler below returns a list of `count` finite floats. A local Mersenne Twister generator and one standard-library distribution are initialized per call, so batch generation avoids repeated Python calls. The same seed and parameters reproduce the sequence with the same C++ implementation; exact streams may differ across platforms. Invalid or nonfinite parameters raise `ValueError`. A nonfinite generated value raises `OverflowError`. The output work is $O(\text{count})$, with distribution-specific draw cost.

```

normal_sample(count, seed)
student_t_sample(count, df, seed)
chi_square_sample(count, df, seed)
f_sample(count, df1, df2, seed)
uniform_sample(count, a, b, seed)
exponential_sample(count, rate, seed)
gamma_sample(count, shape, scale, seed)
weibull_sample(count, shape, scale, seed)
cauchy_sample(count, location, scale, seed)
lognormal_sample(count, mu, sigma, seed)
laplace_sample(count, location, scale, seed)
beta_sample(count, alpha, beta, seed)

import mathbr
d = mathbr.distributions
print(d.normal_sample(5, seed=42))
print(d.gamma_sample(5, shape=2, scale=3, seed=42))
print(d.lognormal_sample(5, mu=0, sigma=1, seed=42))
print(d.laplace_sample(5, location=0, scale=1, seed=42))
print(d.beta_sample(5, alpha=2, beta=3, seed=42))

```

`laplace_sample` uses a random sign and exponential magnitude. `beta_sample` forms a ratio of independent gamma variates after scaling them by their maximum, avoiding overflow in their sum. Extreme beta shapes can cause both gamma variates to underflow to zero; this raises `OverflowError`.

8.10 Geometric and discrete uniform

The geometric distribution counts the trial on which the first success occurs, so its support is $1, 2, \dots$ and its success probability must lie in $(0, 1]$. Its PMF is $p(1 - p)^{k-1}$, and its CDF is $1 - (1 - p)^k$. The discrete uniform distribution has inclusive integer support from a through b , with $a \leq b$. Its PMF is $1/(b - a + 1)$. All quantiles return the smallest supported integer with CDF at least q , where q lies in $[0, 1]$. The geometric PPF returns infinity at $q=1$ unless $p=1$. PMF and CDF are constant-time formulas; the geometric PPF uses logarithms plus a boundary correction.

```
geometric_pmf(k, p)          geometric_cdf(k, p)
geometric_ppf(q, p)
discrete_uniform_pmf(k, a, b)
discrete_uniform_cdf(k, a, b)
discrete_uniform_ppf(q, a, b)

import mathbr
d = mathbr.distributions
print(d.geometric_pmf(3, 0.25), d.geometric_ppf(0.5, 0.25))
print(d.discrete_uniform_cdf(2, 1, 3))
print(d.discrete_uniform_ppf(0.5, 1, 3))
```

8.11 Exponential, Weibull, Laplace, Cauchy, and log-normal

These families expose PDF, CDF, and PPF functions. The exponential uses a positive rate; Weibull uses positive shape and scale; Laplace and Cauchy use finite location and positive scale; log-normal uses finite underlying-normal mean `mu` and positive standard deviation `sigma`. Inputs `x` must be finite and PPF probabilities must lie in $[0, 1]$.

```
exponential_pdf(x, rate)     exponential_cdf(x, rate)
exponential_ppf(p, rate)
weibull_pdf(x, shape, scale) weibull_cdf(x, shape, scale)
weibull_ppf(p, shape, scale)
laplace_pdf(x, location, scale) laplace_cdf(x, location, scale)
laplace_ppf(p, location, scale)
cauchy_pdf(x, location, scale) cauchy_cdf(x, location, scale)
cauchy_ppf(p, location, scale)
lognormal_pdf(x, mu, sigma) lognormal_cdf(x, mu, sigma)
lognormal_ppf(p, mu, sigma)
```

Exponential, Weibull, and log-normal support nonnegative values; their density and CDF are zero below support. Weibull's density at zero is infinite for shape below one, $1/\text{scale}$ at shape one, and zero above shape one. The Cauchy distribution has no finite mean or variance. PPF endpoints return support boundaries or infinities as appropriate.

```
import mathbr
d = mathbr.distributions
print(d.exponential_cdf(1.0, rate=2.0))
print(d.weibull_ppf(0.5, shape=2.0, scale=3.0))
print(d.laplace_pdf(0.0, location=0.0, scale=1.0))
print(d.cauchy_cdf(0.0, location=0.0, scale=1.0))
print(d.lognormal_ppf(0.5, mu=0.0, sigma=1.0))
```

8.12 Gamma and Beta

The gamma distribution uses positive `shape` and `scale`, and the beta distribution uses positive `alpha` and `beta` shape parameters. Their CDFs use the incomplete gamma and beta routines already used by the chi-squared and F distributions. Quantiles use bounded bisection and may be slower than the corresponding density or CDF calls.

```
gamma_pdf(x, shape, scale) gamma_cdf(x, shape, scale)
gamma_ppf(p, shape, scale)
beta_pdf(x, alpha, beta) beta_cdf(x, alpha, beta)
beta_ppf(p, alpha, beta)
```

Gamma support is nonnegative and beta support is $[0, 1]$. A density may be infinite at a support boundary when its corresponding shape is below one. Quantile probabilities must lie in $[0, 1]$.

```
import mathbr
d = mathbr.distributions
print(d.gamma_cdf(1.0, shape=1.0, scale=2.0))
print(d.gamma_ppf(0.5, shape=2.0, scale=3.0))
print(d.beta_pdf(0.5, alpha=2.0, beta=2.0))
print(d.beta_ppf(0.95, alpha=2.0, beta=5.0))
```

8.13 Pareto and Dirichlet

Pareto type I has positive `shape` and lower support bound `scale`. Its density is $\alpha x_m^\alpha / x^{\alpha+1}$ for $x \geq x_m$. The CDF and PPF use logarithmic forms; the PPF returns the lower bound at zero and infinity at one. Seeded sampling draws an exponential log-ratio for each result and raises `OverflowError` if a sampled value is not finite.

```
pareto_pdf(x, shape, scale) pareto_cdf(x, shape, scale)
pareto_ppf(p, shape, scale)
pareto_sample(count, shape, scale, seed)
```

The Dirichlet density is defined here only on the interior of the simplex: every component of `x` and `alpha` must be finite and positive, both vectors must have the same length of at least two, and `x` must sum to one within 10^{-12} . The log-density uses log-gamma terms in $O(c)$ work for c categories. Seeded sampling draws independent gamma values, scales by their maximum, and normalizes each row. This avoids overflow in the sum; extreme gamma underflow or overflow raises `OverflowError`.

```
dirichlet_logpdf(x, alpha) dirichlet_pdf(x, alpha)
dirichlet_sample(count, alpha, seed)

import mathbr
d = mathbr.distributions
print(d.pareto_cdf(2, shape=2, scale=1))
print(d.pareto_sample(5, shape=3, scale=1, seed=42))
print(d.dirichlet_pdf([0.2, 0.3, 0.5], [1, 1, 1]))
print(d.dirichlet_sample(3, [2, 3, 5], seed=42))
```

8.14 Closed-form distribution fitting

Three maximum-likelihood fits require one pass over the sample and constant auxiliary memory. `normal_fit` returns `[mean, sigma]` with $\hat{\sigma}^2 = n^{-1} \sum_i (x_i - \bar{x})^2$; the data must be finite and varying. `exponential_fit` returns rate $1/\bar{x}$ for nonnegative observations with positive mean. `poisson_fit` returns the mean nonnegative integer count, including zero when all counts are zero. Invalid samples raise `ValueError`. These fits do not return uncertainty estimates.

```

normal_fit(observations)
exponential_fit(observations)
poisson_fit(observations)

import mathbr
d = mathbr.distributions
print(d.normal_fit([1, 2, 3, 4]))
print(d.exponential_fit([0, 1, 2, 3]))
print(d.poisson_fit([0, 1, 2, 3]))

```

8.15 Multivariate normal and Mahalanobis distance

`MultivariateNormal(mean, covariance)` accepts a finite mean vector and a matching finite, symmetric positive-definite covariance matrix. Construction computes a Cholesky factor once in $O(p^3)$ time for p dimensions. Density, log-density, and Mahalanobis distance use triangular solves in $O(p^2)$ time per observation, avoiding an explicit inverse. Batch methods reuse the factor and a work vector. `sample(count, seed)` returns row-major draws using the cached factor and a local seeded generator. Nonfinite generated values raise `OverflowError`; invalid shapes or covariance raise `ValueError`.

```

MultivariateNormal(mean, covariance)
logpdf(x)                pdf(x)
mahalanobis_distance(x)
logpdf_batch(data)       pdf_batch(data)
sample(count, seed)
mathbr.statistics.mahalanobis_distance(x, mean, covariance)

```

The scalar statistics convenience function factors the covariance on every call. Reuse a `MultivariateNormal` object for repeated evaluations. The log-density is $-\frac{p}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu)$, evaluated through the Cholesky factor.

```

import mathbr
m = mathbr.distributions.MultivariateNormal(
    [0, 0], [[1, 0.5], [0.5, 2]])
print(m.logpdf([1, 2]), m.mahalanobis_distance([1, 2]))
print(m.logpdf_batch([[0, 0], [1, 2]]))
print(m.sample(3, seed=42))
print(mathbr.statistics.mahalanobis_distance(
    [1, 2], [0, 0], [[1, 0.5], [0.5, 2]]))

```

9 Gradient-descent models

9.1 LinearRegression

`LinearRegression(n_features)` trains a linear model by full-batch MSE gradient descent. `n_features` must be positive. `fit(X, y, lr=0.01, epochs=1000)` accepts nonempty, matching data, rows of the declared width, positive finite learning rate, and positive epoch count. Invalid input raises `ValueError`. The fit returns `None`.

```

predict(x)          -> float      predict_batch(X) -> list[float]
get_weights()       -> list[float] get_bias()        -> float
trained()           -> bool

```

Prediction rows must have the declared width. Initial zero coefficients can be queried before training. This model supplies no coefficient inference; feature scale affects convergence.

```
import mathbr
m = mathbr.LinearRegression(1)
m.fit([[0.0], [1.0], [2.0]], [1.0, 3.0, 5.0], lr=0.1)
print(m.get_weights(), m.get_bias(), m.predict([3.0]))
```

9.2 LogisticRegression

`LogisticRegression(n_features)` uses full-batch gradient descent for binary log loss. The constructor and `fit(X, y, lr=0.01, epochs=1000)` have the same shape and tuning constraints as `LinearRegression`, while labels must be 0 or 1. Fit returns `None`. Invalid inputs raise `ValueError`.

```
predict_proba(x)          -> float
predict_proba_batch(X)    -> list[float]
predict(x, threshold=0.5) -> int
get_weights() -> list[float]  get_bias() -> float
trained() -> bool
```

`threshold` must be finite in $[0, 1]$. The class does not report standard errors, calibration metrics, or classification diagnostics.

```
import mathbr
m = mathbr.LogisticRegression(1)
m.fit([[-2.0], [-1.0], [1.0], [2.0]], [0, 0, 1, 1],
      lr=0.2, epochs=500)
print(m.predict_proba([1.0]), m.predict([1.0]))
```

10 Least squares

10.1 OLS

`OLS(n_features)` includes an intercept and solves ordinary least squares by Householder QR decomposition. `fit(X, y)` requires finite, full-rank data, rows of the declared positive feature width, and more observations than coefficients. It returns `None`. Invalid input or rank deficiency raises `ValueError`.

```
predict(x)          -> float
predict_batch(X)    -> list[float]
coefficients()      -> list[float]  # intercept, then slopes
standard_errors()   -> list[float]
t_statistics()       -> list[float]
p_values()           -> list[float]
confidence_intervals(level=0.95) -> list[list[float]]
r_squared()          -> float
adjusted_r_squared() -> float
f_statistic()        -> float
residual_variance()  -> float
degrees_of_freedom() -> int
trained()            -> bool
```

Fitted results and prediction before fit raise `RuntimeError`. The p-values are two-sided and use Student-t with residual degrees of freedom. Intervals use the same reference distribution; the level must be in $(0, 1)$. The overall F-statistic compares against an intercept-only model. Zero residual variance makes t/F inference undefined and raises `ValueError`. Classical standard errors assume independent, homoscedastic errors; exact small-sample t/F inference additionally assumes Gaussian errors. Robust errors are absent.

```

import mathbr
m = mathbr.OLS(1)
m.fit([[0.0], [1.0], [2.0], [3.0]], [1.0, 3.0, 5.0, 8.0])
print(m.coefficients(), m.standard_errors(), m.r_squared())
print(m.t_statistics(), m.p_values())
print(m.confidence_intervals(), m.f_statistic())

```

10.2 WLS

`WLS(n_features)` inherits every OLS result and prediction method above, including p-values, confidence intervals, adjusted R^2 , and the overall F-statistic. Its fit method takes `X`, `y`, and `weights`. It requires one strictly positive finite weight per observation and returns `None`. Invalid weights raise `ValueError`. The residual variance and R^2 are weighted. Classical WLS inference assumes independent Gaussian errors and weights proportional to inverse error variance.

```

import mathbr
m = mathbr.WLS(1)
m.fit([[0.0], [1.0], [2.0]], [0.0, 1.0, 4.0],
      [1.0, 1.0, 0.1])
print(m.coefficients(), m.predict([3.0]))

```

11 Regression diagnostics

The `mathbr.diagnostics` submodule computes information criteria from a supplied maximized log likelihood and parameter count, or diagnostics from ordered residuals. It does not fit a likelihood model automatically. The parameter count should include all estimated parameters according to the model-comparison convention you use.

```

aic(log_likelihood, n_parameters)
bic(log_likelihood, n_parameters, n_observations)
hqic(log_likelihood, n_parameters, n_observations)
residual_standard_error(residuals, n_parameters)
durbin_watson(residuals)
jarque_bera_statistic(residuals)
jarque_bera_p_value(residuals)

```

AIC is $-2\ell + 2k$, BIC is $-2\ell + k \log n$, and HQIC is $-2\ell + 2k \log \log n$. Residual standard error is $\sqrt{\sum e_i^2 / (n - k)}$. Durbin–Watson is $\sum_{i=2}^n (e_i - e_{i-1})^2 / \sum e_i^2$; residual order matters. Jarque–Bera uses sample skewness and excess kurtosis and its p-value uses the asymptotic chi-squared distribution with two degrees of freedom. These statistics do not establish model validity on their own.

```

import mathbr
d = mathbr.diagnostics
residuals = [-2.0, -1.0, 0.0, 1.0, 2.0]
print(d.aic(-10.0, 2), d.bic(-10.0, 2, 5))
print(d.hqic(-10.0, 2, 5))
print(d.residual_standard_error(residuals, 2))
print(d.durbin_watson(residuals))
print(d.jarque_bera_statistic(residuals),
      d.jarque_bera_p_value(residuals))

```

12 Regularized regression

The constructor signatures are:

```
Ridge(n_features, alpha=1.0, max_iter=1000, tol=1e-8)
Lasso(n_features, alpha=1.0, max_iter=1000, tol=1e-8)
ElasticNet(n_features, alpha=1.0, l1_ratio=0.5,
            max_iter=1000, tol=1e-8)
```

Ridge uses an L2 penalty, Lasso uses L1, and ElasticNet mixes them. Feature count and iteration limit must be positive; `alpha` must be finite and nonnegative, `tol` finite and positive, and `l1_ratio` in $[0, 1]$. Invalid settings raise `ValueError`.

```
fit(X, y)          -> None          predict(x)         -> float
predict_batch(X)   -> list[float]
coefficients()     -> list[float]  intercept()          -> float
iterations()       -> int          converged()             -> bool
trained()          -> bool
```

Fit requires finite, nonempty rows and targets with matching dimensions; invalid input raises `ValueError`. Fitted results before fit raise `RuntimeError`. The intercept is unpenalized. Features are not standardized, so their scales affect the penalty. There are no standard errors; always check `converged()`.

```
import mathbr
m = mathbr.ElasticNet(1, alpha=0.1, l1_ratio=0.5)
m.fit([[-1.0], [0.0], [1.0]], [-2.0, 0.0, 2.0])
print(m.intercept(), m.coefficients(), m.converged())
```

13 Econometrics

13.1 IV2SLS

`IV2SLS(n_exog, n_endog, n_instruments)` specifies counts of exogenous regressors, endogenous regressors, and excluded instruments. `fit(exog, endog, instruments, y)` takes aligned, finite matrices and targets and returns `None`. There must be sufficient instruments and full-rank stages; invalid inputs raise `ValueError`.

```
predict(exog, endog)      -> float
coefficients()             -> list[float]
first_stage_r_squared()    -> list[float]
trained()                  -> bool
```

Coefficients are ordered intercept, exogenous slopes, then endogenous slopes. Fitted results before fit raise `RuntimeError`. Instrument relevance and exclusion must be justified externally. First-stage R^2 is not a weak-instrument test; this class does not provide valid 2SLS standard errors.

```
import mathbr
m = mathbr.IV2SLS(0, 1, 1)
m.fit([], [], [], [], [], [[0], [2], [1], [4], [3]],
      [[0], [1], [2], [3], [4]], [1, 5, 3, 9, 7])
print(m.coefficients(), m.first_stage_r_squared())
```

13.2 FixedEffects

`FixedEffects(n_features)` estimates within-entity slopes. `fit(X, y, entity_ids)` takes finite rows, targets, and aligned integer entity IDs, and returns `None`. Features without within-entity variation cannot be identified; bad dimensions or rank deficiency raise `ValueError`.

```
predict(x, entity_id)      -> float
coefficients()             -> list[float]  # within slopes
entity_intercept(entity_id) -> float
within_r_squared()         -> float
trained()                  -> bool
```

An unknown entity for prediction raises `ValueError`; fitted results before fit raise `RuntimeError`. The model has no time effects or clustered standard errors.

```
import mathbr
m = mathbr.FixedEffects(1)
m.fit([[0], [1], [2], [0], [1], [2]],
      [1, 3, 5, 10, 12, 14], [10, 10, 10, 20, 20, 20])
print(m.coefficients(), m.entity_intercept(20))
```

14 Time series

`AR(lags)` fits one series; `VAR(n_series, lags)` fits multiple series. Lag counts and series count must be positive. `AR.fit(observations)` takes a finite sequence; `VAR.fit(observations)` takes finite rows with `n_series` columns. Both need enough data and full-rank lag matrices; otherwise they raise `ValueError`. Fit returns `None`.

```
AR: coefficients() -> list[float]
    forecast(steps) -> list[float]
    trained()      -> bool
VAR: coefficients() -> list[list[float]]
    forecast(steps) -> list[list[float]]
    residual_covariance() -> list[list[float]]
    trained()      -> bool
```

Forecast steps must be positive; fitted results before fit raise `RuntimeError`. AR coefficients begin with the intercept. Each VAR equation has intercept, then lag-1 series, lag-2 series, and so on. Neither class chooses lags, tests stationarity, or provides forecast intervals.

```
import mathbr
ar = mathbr.AR(1)
ar.fit([1, 2, 4, 8, 16, 32])
print(ar.coefficients(), ar.forecast(2))
```

15 Time-series diagnostics

The `mathbr.time_series_diagnostics` submodule computes direct, biased sample autocorrelation, partial autocorrelation by Durbin–Levinson recursion, and the Ljung–Box portmanteau statistic. Inputs must contain at least two finite, varying observations. The largest lag must be smaller than the series length.

```
acf(x, max_lag)
pacf(x, max_lag)
ljung_box(x, lags)
```

`acf` and `pacf` return lists from lag zero through `max_lag`, with lag zero equal to one. Direct ACF costs $O(nk)$ for k requested lags; PACF adds $O(k^2)$ recursion. `ljung_box` returns a `LjungBoxResult` with read-only `statistic`, `p_value`, and `lags`. Its p-value uses a chi-squared reference with degrees of freedom equal to the requested lags, with no correction for fitted model parameters. The approximation should be interpreted cautiously for short series.

```
import mathbr
d = mathbr.time_series_diagnostics
series = [1.0, 2.0, 3.0, 2.0, 1.0, 0.0]
print(d.acf(series, 2))
print(d.pacf(series, 2))
result = d.ljung_box(series, 2)
print(result.statistic, result.p_value, result.lags)
```

16 Volatility

`GARCH(max_iter=2000, tol=1e-7, arch_only=False)` fits `GARCH(1,1)`. Set `arch_only=True` to fix beta at zero. `ARCH(max_iter=2000, tol=1e-7)` is the `ARCH(1)` subclass. Iteration limit and tolerance must be positive. `fit(observations)` requires at least 20 finite observations with positive variance, returns `None`, and raises `ValueError` otherwise.

```
mean()           -> float
omega()          -> float
alpha()          -> float
beta()           -> float
log_likelihood() -> float
conditional_variance() -> list[float]
forecast_variance(steps) -> list[float]
converged()      -> bool
trained()        -> bool
```

`forecast_variance` requires positive steps. Fitted results before fit raise `RuntimeError`. These constant-mean, symmetric Gaussian quasi-likelihood models have no coefficient standard errors, asymmetric shocks, or non-Gaussian innovations. Check `converged()` before interpreting parameters.

```
import mathbr
m = mathbr.GARCH()
m.fit([-2.0, -1.0, 0.5, 1.0, 2.0] * 20)
print(m.omega(), m.alpha(), m.beta())
print(m.forecast_variance(3), m.converged())
```

17 Conjugate Bayesian updating

The `mathbr.bayesian` submodule provides two closed-form posterior updates. Beta–Binomial takes a positive beta prior and independent Bernoulli trials with a shared success probability. Normal–Normal takes a normal prior for a mean and independent normal observations with a *known* common observation standard deviation. These assumptions are inputs to the analysis, not checked by the code.

```
beta_binomial_update(alpha, beta, successes, trials)
normal_normal_update(prior_mean, prior_sd,
                     observation_sd, observations)
```

```

BetaPosterior.mean()
BetaPosterior.credible_interval(level=0.95)
NormalPosterior.credible_interval(level=0.95)

```

The beta posterior exposes its shape parameters and mean. The normal posterior exposes its mean and standard deviation. Both interval methods return equal-tailed bounds. The level must lie strictly between zero and one. These are posterior probability intervals under their stated models, rather than frequentist coverage guarantees.

```

import mathbr
b = mathbr.bayesian
rate = b.beta_binomial_update(1, 1, successes=7,
                              trials=10)
print(rate.alpha, rate.beta, rate.mean())
print(rate.credible_interval(0.95))
normal = b.normal_normal_update(0, 1, 1, [1, 3])
print(normal.mean, normal.standard_deviation)
print(normal.credible_interval(0.95))

```

The uniform Beta(1,1) prior becomes Beta(8,4) after seven successes in ten trials, giving a posterior mean of $8/12 \approx 0.667$. The normal example has prior mean zero and two observations with mean two. With all standard deviations equal to one, its posterior mean is $4/3$ and its posterior standard deviation is $1/\sqrt{3}$. The intervals express uncertainty conditional on these prior and likelihood choices.

18 Right-censored survival analysis

The `mathbr.survival` submodule accepts nonnegative, finite observation times and binary event indicators: 1 means the event occurred, 0 means right censored. It assumes independent censoring and no delayed entry. A group indicator of 0 or 1 is required for the log-rank test.

```

kaplan_meier(times, events)
log_rank_test(times, events, groups)

```

`kaplan_meier` returns a `KaplanMeierResult` with read-only `times`, `survival`, `at_risk`, and `events` arrays at each distinct observed time. The survival estimate changes only at event times. Events at a tied time count before all subjects at that time leave the risk set. `log_rank_test` returns a `LogRankResult` with read-only `statistic` and `p_value`. It compares two groups using pooled risk sets and a chi-squared reference with one degree of freedom. It needs events and a positive test variance. These functions do not account for covariates, competing risks, or left truncation.

```

import mathbr
s = mathbr.survival
curve = s.kaplan_meier([1, 2, 2, 3], [1, 0, 1, 1])
print(curve.times, curve.survival)
print(curve.at_risk, curve.events)
comparison = s.log_rank_test([1, 1, 2, 2],
                             [1, 1, 1, 1], [0, 0, 1, 1])
print(comparison.statistic, comparison.p_value)

```

In the Kaplan–Meier example, the survival estimates after times 1, 2, and 3 are 0.75, 0.50, and 0. The subject censored at time 2 remains in the risk set for events at that time, then leaves it. The log-rank p-value compares event timing between the two example groups under its null model; it does not quantify an effect size.

19 Worked examples

These examples use small data sets so that the inputs and results are easy to inspect. Each code block can be run independently after installing `mathbr`.

19.1 Machine learning: fit, predict, and evaluate held-out data

Keep the final two observations out of fitting. Fit an OLS line on the first four observations, predict the held-out targets, and then calculate two error measures. This separates estimation from evaluation, even though the toy data are much smaller than a real study.

```
import math
import mathbr
X_train = [[0.0], [1.0], [2.0], [3.0]]
y_train = [1.0, 3.0, 5.0, 7.0]
X_test = [[4.0], [5.0]]
y_test = [9.0, 12.0]
model = mathbr.OLS(1)
model.fit(X_train, y_train)
predicted = model.predict_batch(X_test)
rmse = mathbr.evaluation.rmse(y_test, predicted)
mape = mathbr.evaluation.mape(y_test, predicted)
print("intercept and slope:", model.coefficients())
print("held-out predictions:", predicted)
print("RMSE:", rmse, "MAPE fraction:", mape)
assert all(math.isclose(a, b) for a, b in
            zip(predicted, [9.0, 11.0]))
assert math.isclose(rmse, math.sqrt(0.5))
```

The fitted line has intercept 1 and slope 2. Its first held-out prediction is exact; the second misses by 1, so RMSE is $\sqrt{(0^2 + 1^2)/2} \approx 0.707$. MAPE is $(0/9 + 1/12)/2 \approx 0.0417$, a fraction equivalent to about 4.17 percent. Do not use the test targets to tune the model and then report the same error as an untouched test result.

19.2 Machine learning: compare regularization strengths

Fit the same one-feature data with three penalties. Their slope estimates differ because the penalties shrink coefficients differently. Check convergence before using a penalized fit.

```
import mathbr
X = [[-1.0], [0.0], [1.0]]
y = [-2.0, 0.0, 2.0]
for name, model in [
    ("Ridge", mathbr.Ridge(1, alpha=1.0)),
    ("Lasso", mathbr.Lasso(1, alpha=0.5)),
    ("Elastic Net", mathbr.ElasticNet(1, alpha=1.0))
]:
    model.fit(X, y)
    print(name, model.coefficients(), model.converged())
```

These coefficients are not directly comparable across differently scaled features because automatic standardization is absent. No standard errors or p-values are available for these penalized models.

19.3 Machine learning: binary probabilities and decisions

Fit a binary classifier, inspect probabilities, and apply a decision threshold. A higher threshold changes the class decision, not the fitted probability.

```
import mathbr
X = [[-2.0], [-1.0], [1.0], [2.0]]
y = [0, 0, 1, 1]
model = mathbr.LogisticRegression(1)
model.fit(X, y, lr=0.2, epochs=500)
for row in [[-1.0], [0.0], [1.0]]:
    print(row, model.predict_proba(row),
          model.predict(row, threshold=0.7))
```

These predictions use the fitted sample. Assess generalization with evaluation metrics on held-out observations. The class does not automatically calibrate its probabilities.

19.4 Statistics: inspect an OLS fit

Estimate an intercept and slope, then calculate residuals and an in-sample RMSE using the loss submodule. The standard errors returned by OLS are classical, not heteroskedasticity-robust.

```
import mathbr
X = [[0.0], [1.0], [2.0], [3.0]]
y = [1.0, 3.0, 5.0, 8.0]
model = mathbr.OLS(1)
model.fit(X, y)
fitted = model.predict_batch(X)
residuals = [actual - predicted
              for actual, predicted in zip(y, fitted)]
print("coefficients:", model.coefficients())
print("standard errors:", model.standard_errors())
print("two-sided p-values:", model.p_values())
print("95% confidence intervals:",
      model.confidence_intervals())
print("R squared:", model.r_squared())
print("adjusted R squared:", model.adjusted_r_squared())
print("residuals:", residuals)
print("in-sample RMSE:", mathbr.losses.rmse(y, fitted))
```

The p-values and intervals use classical Gaussian-error assumptions; they are not robust to arbitrary heteroskedasticity. The RMSE above is in-sample and is not an out-of-sample error estimate.

19.5 Statistics: downweight one observation with WLS

Use a smaller weight for an observation judged less precise. Weight choice must reflect a defensible error-variance assumption for classical WLS standard errors to be meaningful.

```
import mathbr
X = [[0.0], [1.0], [2.0], [3.0]]
y = [0.0, 1.0, 2.0, 8.0]
ols = mathbr.OLS(1)
ols.fit(X, y)
wls = mathbr.WLS(1)
```

```
wls.fit(X, y, [1.0, 1.0, 1.0, 0.1])
print("OLS:", ols.coefficients())
print("WLS:", wls.coefficients())
```

The lower final weight changes the estimated line. WLS weights must always be strictly positive and finite.

19.6 Econometrics: instrumental variables

Separate the observed endogenous regressor from its excluded instrument. The first-stage R^2 describes fit, but does not establish instrument validity or provide a weak-instrument test.

```
import mathbr
z = [[0.0], [1.0], [2.0], [3.0], [4.0]]
endog = [[0.0], [2.0], [1.0], [4.0], [3.0]]
y = [1.0, 5.0, 3.0, 9.0, 7.0]
model = mathbr.IV2SLS(0, 1, 1)
model.fit([[] for _ in y], endog, z, y)
print("intercept and IV slope:", model.coefficients())
print("first-stage R squared:",
      model.first_stage_r_squared())
print("prediction at endog=5:", model.predict([], [5.0]))
```

The class does not verify the exclusion restriction and does not report valid 2SLS standard errors. A causal interpretation needs a credible identification argument outside the code.

19.7 Econometrics: entity fixed effects

The two entities have different intercepts but share a within-entity slope. The fitted slope uses changes within each entity rather than differences in their average levels.

```
import mathbr
X = [[0], [1], [2], [0], [1], [2]]
y = [1, 3, 5, 10, 12, 14]
entity_ids = [10, 10, 10, 20, 20, 20]
model = mathbr.FixedEffects(1)
model.fit(X, y, entity_ids)
print("within slope:", model.coefficients())
print("entity 10 intercept:", model.entity_intercept(10))
print("entity 20 intercept:", model.entity_intercept(20))
print("entity 20 prediction:", model.predict([3], 20))
```

Prediction requires an entity seen during fitting. The model has no time effects or entity-clustered standard errors.

19.8 Time series: two-variable forecasting

Generate a small two-series history from a known lag-one system, fit VAR(1), and forecast the next two rows. This demonstrates coefficient ordering and recursive forecasts.

```
import mathbr
history = [[1.0, 2.0]]
for _ in range(20):
    x, y = history[-1]
    history.append([1.0 + 0.5*x + 0.2*y,
```

```

-1.0 + 0.3*x + 0.4*y])
model = mathbr.VAR(n_series=2, lags=1)
model.fit(history)
print("equation coefficients:", model.coefficients())
print("next two rows:", model.forecast(2))
print("residual covariance:",
      model.residual_covariance())

```

VAR does not test stationarity or select the lag order. Its forecasts are point forecasts without intervals.

19.9 Volatility: compare ARCH and GARCH forecasts

Fit both volatility recursions on the same observations. Their first variance forecasts use the last observed shock; later horizons use expected future shocks.

```

import mathbr
observations = [-2.0, -1.0, 0.5, 1.0, 2.0] * 20
for name, model in [("ARCH", mathbr.ARCH()),
                    ("GARCH", mathbr.GARCH())]:
    model.fit(observations)
    print(name, "converged:", model.converged())
    print("parameters:", model.omega(),
          model.alpha(), model.beta())
    print("variance forecast:",
          model.forecast_variance(3))

```

These are Gaussian quasi-likelihood fits. The optimizer may fail to converge, and the classes provide no parameter standard errors or forecast intervals.

19.10 Probability: use a quantile as a cutoff

A standard-normal or Student-t quantile can be used as a mathematical cutoff. Computing a cutoff does not by itself validate the assumptions of a statistical test.

```

import mathbr
d = mathbr.distributions
normal_cutoff = d.normal_ppf(0.975)
t_cutoff = d.student_t_ppf(0.975, df=10.0)
print("normal cutoff:", normal_cutoff)
print("Student-t cutoff:", t_cutoff)
print("normal CDF at cutoff:",
      d.normal_cdf(normal_cutoff))

```

20 API boundaries

`mathbr.version` is the version string. `mathbr._RegularizedRegression` is an exposed binding base class for Ridge, Lasso, and ElasticNet; it has no public constructor. Its methods are the inherited regularized methods listed above. This educational library has no formula interface, DataFrame integration, robust covariance, or general result summary yet. For exact signatures and further notes, consult `docs/API_REFERENCE.md` in the source tree.